Editing Your GitHub Pages Site on a Laptop

A step-by-step guide: install Git, clone your repository, make changes, and push them back to GitHub. Replace **YOUR-USERNAME** and **YOUR-REPO** with your real GitHub username and repository name (for a user site this is usually *YOUR-USERNAME.github.io*).

The Big Picture

Your website lives in a GitHub repository. You will make a local copy on the laptop (**clone**), edit files, record the edits (**commit**), and send them to GitHub (**push**). GitHub Pages then rebuilds the live site automatically, usually within a minute or two.

Term	Meaning
clone	Download a full copy of the repo to your laptop
add / stage	Pick which changed files go in the next save point
commit	Save a snapshot of your staged changes locally
push	Upload your commits to GitHub
pull	Download new changes from GitHub to your laptop

Step 1: Install Git on the Laptop

Ubuntu / Lubuntu / Debian:

```
sudo apt update
sudo apt install git
```

Windows: download Git for Windows from git-scm.com and install with the default options. Then use "Git Bash" for the commands in this guide.

macOS: open Terminal and run `git --version`; it will offer to install the tools if missing.

Confirm it worked:

```
git --version
```

Step 2: Tell Git Who You Are (One Time Only)

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

Note: Use the same email that is attached to your GitHub account so commits are linked to your profile.

Step 3: Set Up Authentication with GitHub

GitHub no longer accepts your account password for pushing. Pick **one** of these two methods.

Option A: HTTPS with a Personal Access Token (simplest)

1. On github.com, click your profile picture, then **Settings**.
2. Scroll to **Developer settings**, then **Personal access tokens**, then **Tokens (classic)**.
3. Click **Generate new token (classic)**. Give it a name, choose an expiration, and tick the **repo** scope.
4. Click **Generate token** and copy it immediately. GitHub shows it only once.
5. When Git asks for a password during your first push, paste the token (your username is your GitHub username).

To avoid retyping the token each time, tell Git to remember it:

```
git config --global credential.helper store
```

Note: The "store" helper saves the token in plain text in your home folder. On a shared laptop, use `cache` instead, which keeps it in memory temporarily.

Option B: SSH key (no password prompts afterward)

```
ssh-keygen -t ed25519 -C "you@example.com"
# press Enter to accept defaults
cat ~/.ssh/id_ed25519.pub
```

Copy the full output. On GitHub go to **Settings, SSH and GPG keys, New SSH key**, paste it, and save. Then test the connection:

```
ssh -T git@github.com
```

If you use SSH, clone with the SSH address (`git@github.com:...`) in the next step.

Step 4: Clone Your Repository to the Laptop

On your repo's GitHub page, click the green **Code** button and copy the HTTPS (or SSH) address. In a terminal, move to the folder where you want the project, then clone:

```
cd ~/Documents

# HTTPS:
git clone https://github.com/YOUR-USERNAME/YOUR-REPO.git

# or SSH:
git clone git@github.com:YOUR-USERNAME/YOUR-REPO.git

cd YOUR-REPO
```

This creates a folder named after the repo containing all your site files. You only clone once per laptop.

Step 5: Pull the Latest Changes Before You Start

Always do this at the start of every work session, especially if you also edit the site from another computer or directly on github.com. It prevents conflicts.

```
git pull
```

Step 6: Make Changes to the Website

Open the folder in any editor (VS Code, Geany, Notepad++, nano, etc.) and edit your files: `index.html`, CSS, images, Markdown pages, and so on.

To preview locally before publishing, the easiest way is a tiny built-in web server. From the repo folder:

```
python3 -m http.server 8000
```

Then open `http://localhost:8000` in your browser. Press Ctrl+C in the terminal to stop it.

Note: If your site uses Jekyll (look for a `_config.yml` file), the plain server above will not process Jekyll templates. Install Jekyll and run `bundle exec jekyll serve` instead, or simply push and check the live site.

Step 7: Review What You Changed

```
git status        # lists changed, new, and deleted files
git diff          # shows the exact line-by-line changes
```

Step 8: Stage, Commit, and Push

Stage the changes (the dot means "everything changed in this folder"):

```
git add .
```

Commit with a short message describing what you did:

```
git commit -m "Update about page and fix header link"
```

Push to GitHub:

```
git push
```

If Git asks for credentials, use your username and Personal Access Token (Option A). With SSH there will be no prompt.

Note: On a brand new repo whose branch has not been linked yet, you may need `git push -u origin main` once. Your branch may be called *master* instead of *main*; check with `git branch`.

Step 9: Confirm the Site Updated

1. Visit your repo on GitHub and check that your commit appears.
2. Click the **Actions** tab to watch the "pages build and deployment" run. A green check means it deployed.
3. Open your live site. Hard refresh with Ctrl+Shift+R if you do not see changes, since browsers cache pages.

Everyday Workflow (Cheat Sheet)

```
cd ~/Documents/YOUR-REPO
git pull                      # get latest
# ...edit your files...
git status                    # see what changed
git add .                     # stage
git commit -m "What I did"    # save snapshot
git push                      # publish
```

Troubleshooting

Problem	Fix
Authentication failed on push	You used your account password. Use a Personal Access Token instead, or set up SSH.
rejected: non-fast-forward / fetch first	GitHub has changes you lack. Run git pull, then git push again.
Merge conflict after pull	Open the files marked with <<<<<<<, keep the correct text, delete the markers, then git add . and git commit.
Changes not showing on live site	Check the Actions tab for build errors, wait a couple of minutes, and hard refresh the browser.
Wrong files committed	Before pushing: git reset HEAD~1 undoes the last commit but keeps your edits.
Permission denied (publickey)	SSH key not added to GitHub, or you cloned with HTTPS and SSH mixed. Check git remote -v.